Professional Training Guide
JavaScript • TypeScript • Google Apps Script

Beginner-Friendly • Training Ready • Team Share Version

Page11: Topic 11 — Classes & OOP

What is OOP?

OOP means Object-Oriented Programming.

It is a way of writing code using objects and classes.

Class = Class is bluePrint of Object

Object = Object is anything Like A Pen, a laptop, Id Card

Simple Real-Life Example

Think about a Car

A car has:

Properties → color, brand, speed

Actions → start(), stop()

Main OOP Concepts (Very Simple)

Full Simple Example (JavaScript)

```
class Car {

  brand = "BMW";

  start() {
    console.log(this.brand + " Started");
  }

}

const car1 = new Car();

car1.start();
```

Output:

BMW Started

Why this?

this means:Current object

Example: this.brand

Constructor

Constructor runs automatically when object is created.

```
class Car {
```

```
  constructor(brand) {
    this.brand = brand;
  }
}
```

4 Important OOP Pillars

1. Encapsulation

Keeping data + methods together.

```
class Bank {

  balance = 1000;

  showBalance() {
    console.log(this.balance);
  }

}
```

Easy memory trick:

Encapsulation = Wrap data together

2. Inheritance

```
One class uses another class features.

class Animal {
  eat() {
    console.log("Eating");
  }
}

class Dog extends Animal {

}

const d = new Dog();

d.eat();
```

Easy memory trick:

Inheritance = Child gets parent power

3. Polymorphism

Same method, different behavior.

```
class Animal {
  sound() {
    console.log("Animal Sound");
  }
}

class Dog extends Animal {
  sound() {
    console.log("Bark");
  }
}
```

Easy memory trick:

Same function, different output

4. Abstraction

Hide complex things.

Example:

car.start()

You use it without knowing engine details.

Easy memory trick:

Hide complexity

Super Important Keywords

Inheritance Example (Simple)

```
class Animal {

  constructor(name) {
    this.name = name;
  }

}

class Dog extends Animal {

}

const d1 = new Dog("Tommy");

console.log(d1.name);
```

super() Example

```
class Animal {

  constructor(name) {
    this.name = name;
  }

}

class Dog extends Animal {

  constructor(name) {
    super(name);
  }

}
```

Easy memory trick:

super() = Call parent constructor

Golden Rule

OOP helps organize code using classes and objects for scalable and reusable applications.

It is widely used in modern JavaScript, TypeScript, backend systems, and large applications.

Page12: Topic 12 — Modules

Modules mean dividing code into small reusable files.

Modules help split code into multiple files.

Instead of writing everything in one big file, we divide code into small reusable files called modules.

or

Split big code into small files and reuse them.

Easy memory trick:

Module = Separate reusable file

Simple Example

math.js

```
export function add(a, b) {
  return a + b;
}
```

main.js

```
import { add } from "./math.js";

console.log(add(2, 3));
```

Output:

5

Main Module Concepts

In Modules, mainly there are 2 types of exports:

Named Export → Export many things

Default Export → Export one main thing

1. Export

Used to share variables, functions, or classes.

Named Export

```
export const name = "Akhil";

export function greet() {

  console.log("Hello");

}
```

Import Named Exports

```
import { name, greet } from "./user.js";

console.log(name);

greet();
```

2. Default Export

```
One main export per file.

export default function greet() {

  console.log("Hello");

}
```

Import Default Export

```
import greet from "./user.js";

greet();

No {} needed.
```

Final Example:

// export

```
export const name = "Akhil";


// export function

export function greet() {

  console.log("Hello");

}
```

// import

```
import { name, greet } from "./app.js";


console.log(name);
```

greet();

Golden Rule

Modules help split large applications into small reusable files.

They are essential in modern JavaScript and TypeScript development.

Page13: Topic 13 — TypeScript Basics

TypeScript (TS) is superset of JavaScript

Or

It means:

TypeScript = JavaScript + Types

All JavaScript code works in TypeScript

TypeScript adds safety and better tooling

Created by Microsoft.

Basic Types

Example:

// types

```
let name: string = "Akhil";

let age: number = 20;

let isAdmin: boolean = true;
```

// array

```
let nums: number[] = [1, 2, 3];
```

// function

```
function add(a: number, b: number): number {
 return a + b;
}
```

// object

```
let user: { name: string; age: number } = {
 name: "Akhil",
 age: 20
};
```

Golden Rule

TypeScript is JavaScript with type safety.

It helps build cleaner, safer, and scalable modern applications.

Page14: Topic 14 — Array Methods Deep Dive

Array is COntainer Which Ontain Similar data.

Most Important Array Methods

1. push()

Adds value at end.

```
const fruits = ["Apple"];

fruits.push("Mango");

console.log(fruits);
```

Output:

["Apple", "Mango"]

2. pop()

Removes last item.

```
const fruits = ["Apple", "Mango"];

fruits.pop();

console.log(fruits);
```

Output:

["Apple"]

3. shift()

Removes first item.

```
const fruits = ["Apple", "Banana"];

fruits.shift();
```

```
console.log(fruits);
```

4. unshift()

Adds at beginning.

```
const fruits = ["Banana"];

fruits.unshift("Apple");

console.log(fruits);
```

5. slice()

Copies portion of array.

```
const numbers = [1, 2, 3, 4, 5];

console.log(numbers.slice(1, 4));
```

Output:

[2, 3, 4]

⚠ Original array does not change.

6. splice()

Adds/removes items.

```
const fruits = ["Apple", "Banana"];

fruits.splice(1, 0, "Mango");

console.log(fruits);
```

Output:

["Apple", "Mango", "Banana"]

Remove using splice

```
const numbers = [1, 2, 3];

numbers.splice(1, 1);

console.log(numbers);
```

Output:

[1, 3]

7. map()

Transforms every item.

```
const numbers = [1, 2, 3];

const doubled = numbers.map(num => num * 2);

console.log(doubled);
```

Output:

[2, 4, 6]

⚠ Returns new array.

8. filter()

Filters matching items.

```
const ages = [10, 20, 30];

const adults = ages.filter(age => age >= 18);

console.log(adults);
```

Output:

[20, 30]

9. find()

Returns first matching item.

```
const users = [

  { name: "Akhil" },

  { name: "Rahul" }

];

const user = users.find(u => u.name === "Rahul");

console.log(user);
```

Output:

```
{ name: "Rahul" }
```

10. findIndex()

Returns matching index.

```
const numbers = [10, 20, 30];

const index = numbers.findIndex(num => num === 20);

console.log(index);
```

Output:

1

11. forEach()

Loops through array.

```
const fruits = ["Apple", "Mango"];

fruits.forEach(fruit => {

  console.log(fruit);

});
```

Difference → map vs forEach

12. reduce()

Combines values into one result.

```
const numbers = [1, 2, 3, 4];

const total = numbers.reduce((sum, num) => {

  return sum + num;

}, 0);

console.log(total);
```

Output:

10

reduce() Deep Understanding

```
array.reduce((accumulator, current) => {

}, initialValue);
```

13. some()

Checks if at least one matches.

```
const numbers = [1, 2, 3];

console.log(numbers.some(num => num > 2));
```

Output:

true

14. every()

Checks if all match.

```
const numbers = [1, 2, 3];

console.log(numbers.every(num => num > 0));
```

Output:

true

15. includes()

Checks existence.

```
const fruits = ["Apple", "Mango"];

console.log(fruits.includes("Apple"));
```

16. indexOf()

Finds position.

```
const fruits = ["Apple", "Mango"];

console.log(fruits.indexOf("Mango"));
```

Output:

1

17. sort()

Sorts array.

```
const numbers = [3, 1, 2];

numbers.sort();

console.log(numbers);
```

⚠ String sorting by default.

Correct numeric sorting:

```
numbers.sort((a, b) => a - b);
```

18. reverse()

Reverses array.

```
const numbers = [1, 2, 3];

numbers.reverse();

console.log(numbers);
```

19. join()

Converts array into string.

```
const fruits = ["Apple", "Mango"];

console.log(fruits.join(", "));
```

Output:

Apple, Mango

20. flat()

Flattens nested arrays.

```
const arr = [[1, 2], [3, 4]];

console.log(arr.flat());
```

Output:

[1, 2, 3, 4]

Chaining Methods

Very powerful concept.

```
const numbers = [1, 2, 3, 4];

const result = numbers
  .filter(num => num > 2)
  .map(num => num * 2);

console.log(result);
```

Output:

[6, 8]

Immutable vs Mutable Methods

Mutable changes original array.

Most Important Array Methods to Master

map

filter

reduce

find

forEach

some

every

sort

splice

slice

Best Practices

Use map() for transformation

```
const result = arr.map(item => item * 2);
```

Use filter() for filtering

```
const active = users.filter(user => user.active);
```

Avoid unnecessary loops

Modern methods are cleaner.

Golden Rule

Array methods help process data efficiently and cleanly.

They are heavily used in modern JavaScript and TypeScript development.

Page15: Topic 15 — APIs & JSON

What Is an API?

API means:

Application Programming Interface

It allows applications to communicate with each other.

Example:

Website ↔ Server ↔ Database

or

API helps two applications talk to each other.

Example:

Weather app gets weather data from server

Payment app connects with bank

Google Maps shows location data

Simple Meaning

API = A bridge to get or send data

What is JSON?

JSON means JavaScript Object Notation.

It is a lightweight format for store and send data.

JSON Example

```
{
  "name": "Akhil",
  "age": 25,
  "city": "Bangalore"
}
```

Data is stored in key : value format.

Very easy to read.

Why JSON Is Popular

JSON is:

Easy to read

Lightweight

Fast

Supported everywhere

Most APIs return JSON data.

API Request Methods

HTTP Status Codes

Example:

// JSON

```
const user = {
```

name: "Akhil",

age: 20

};

```
// object → JSON
const jsonData = JSON.stringify(user);


// JSON → object
const obj = JSON.parse(jsonData);


console.log(obj);


// API fetch
fetch("https://api.example.com/users")
  .then(res => res.json())
  .then(data => console.log(data));
```

Golden Rule

APIs allow applications to communicate, and JSON is the most common data format used for that communication.

Modern web and mobile applications heavily depend on APIs and JSON.

will upload shortly Below

Topic 16 — Events & Event Handling –

What Are Events?

Events are user actions.

or

Events happen when a user does something on a webpage.

Examples:

Click button

Type in input box

Move mouse

Press keyboard key

What is Event Handling?

Event Handling means:

Running code when event happens.

Click Event Example

<button id="btn">Click</button>

```
document.getElementById("btn")
  .addEventListener("click", function () {

    console.log("Button Clicked");

});
```

Output:

Button Clicked

addEventListener()

Best way to handle events.

```
let btn = document.querySelector("button");

btn.addEventListener("click", () => {
  console.log("Clicked");
});
```

Common Events

Example:

Click Event Example

<button id="btn">Click</button>

```
document.getElementById("btn")
  .addEventListener("click", function () {

    console.log("Button Clicked");

});
```

Output:

Button Clicked

Input Event Example

<input id="name">

```
document.getElementById("name")
  .addEventListener("input", function (e) {

    console.log(e.target.value);

});
```

Easy understanding:

e.target.value = Current input value

Golden Rule

Events make webpages interactive, and event handling allows JavaScript to respond to user actions dynamically.

It is one of the most important frontend JavaScript concepts.

Topic 17 — Closures & Scope

What Is Scope?

Scope defines where variables can be accessed.

Scope means:Where variables can be used.

Easy memory trick:

Scope = Variable access area

Types of Scope

Global Scope

Function Scope

Block Scope

Global Scope

Variable can be used anywhere.

```
let name = "Akhil";

function show() {
  console.log(name);
}
```

Function Scope

Variable works only inside function.

```
function test() {
  let age = 25;
  console.log(age);
}
```

Block Scope

```
let and const work only inside { }

{
  let city = "Bangalore";
}
```

```
var vs let vs const
```

What is Closure?

Closure means:

```
A function remembers variables from its outer function even after outer function finishes.
```

Closure Example

```
function outer() {
  let count = 0;

  return function inner() {
    count++;
    console.log(count);
  };
}

let counter = outer();

counter();
counter();
```

Output

1
2

inner() remembers count variable.

Simple Remember

Scope → Where variable works

Closure → Function remembers outer variables

Real Life Example

Password stored safely inside function

Closures help protect/private data.

Golden Rule

Closures allow functions to remember variables from their outer scope even after execution finishes.

Closures and scope are foundational concepts in modern JavaScript and TypeScript.

Topic 18 — Google Apps Script Services

Google Apps Script Services

Google Apps Script Services help us work with Google products using code.

or

Google Apps Script Services are built-in tools provided by Google to interact with Google products.

Google Apps Script Services help your script work with Google products.

Easy meaning: Services = Ready-made tools from Google

Using services, we can control:

Google Sheets

Gmail

Google Drive

Google Forms

Calendar

Docs

Simple Understanding

Apps Script → Controls Google Services

Example:

Apps Script → Read Google Sheet
Apps Script → Send Gmail
Apps Script → Create Drive Folder

Why Services are Important?

Services help automate work in Google products.

Examples:

Send automatic emails

Update Google Sheets

Create reports

Store files in Drive

Create forms automatically

Types of Google Apps Script Services

Short Remember

SpreadsheetApp → Google Sheets

GmailApp → Gmail

DriveApp → Google Drive

FormApp → Google Forms

DocumentApp → Google Docs

CalendarApp → Google Calendar

Most Popular Services

Common Google Apps Script Services

1. Spreadsheet Service

Used to work with Google Sheets.

```
let sheet = SpreadsheetApp.getActiveSpreadsheet();
```

Example

sheet.getSheetByName("Sheet1");

2. Gmail Service

Used to send emails.

```
GmailApp.sendEmail(
  "test@gmail.com",
  "Hello",
  "Welcome"
);
```

3. Drive Service

Used to access Google Drive files.

DriveApp.getFiles();

4. Form Service

Used to create and manage Google Forms.

FormApp.create("Student Form");

5. Document Service

Used to work with Google Docs.

DocumentApp.create("My Document");

6. Calendar Service

Used to manage Google Calendar events.

CalendarApp.getDefaultCalendar();

Golden Rule

Google Apps Script Services allow automation and integration across Google products and external systems.

They are the foundation of Google Apps Script development.

Topic 19 — Debugging & Console

What is Debugging?

Debugging means finding and fixing errors in code.

When code is not working properly, we use debugging to check the problem.

What is Console?

Console is a tool used to:

Show output

Check values

Find errors

Test code

In browser, open console using:

F12

Right Click → Inspect → Console

```
console.log()
```

Used to print values.

```
console.log("Hello");
```

Check Variable Value

```
let name = "Akhil";

console.log(name);
```

```
console.error()
```

Shows error message.

```
console.error("Something went wrong");
```

```
console.warn()
```

Shows warning message.

```
console.warn("Warning");
```

Using Debugging

```
let a = 10;
let b = 20;

console.log(a + b);
```

We check output step by step.

debugger Keyword

Stops code execution for checking.

```
let x = 5;

debugger;

console.log(x);
```

Short Remember

Debugging → Find & fix errors

Console → Check output and errors

```
console.log() → Print value

console.error() → Show error
```

debugger → Pause code for checking

Most Important Debugging Topics

`console.log`

`console.error`

breakpoints

debugger

try...catch

DevTools

Network tab

stack trace

TypeScript errors

Logger.log

Golden Rule

Debugging is the process of understanding what the code is doing and fixing problems step by step.

Good debugging skills make developers faster and more effective.

Topic 20 — Git & GitHub Basics

What Is Git?

Git is a version control system.

Git is a tool used to track code changes.

It helps developers:

Save code history

Manage projects

Work with teams

Restore old code

Git = Save history of code

What Is GitHub?

url GitHub https://github.com is a platform for storing and sharing code.(Like Facbook, Instagarm, youtube)

GitHub is a website to store Git projects online.

It is used to:

Upload code

Share projects

Work with teams

Backup code

Simple Meaning

Git → Track code

GitHub → Store code online

Common Git Commands

1. git init

Start Git in project.

```
git init
```

2. git status

Check file status.

```
git status
```

3. git add

Add files for save.

```
git add .
```

4. git commit

Save code changes.

```
git commit -m "First commit"
```

5. git push

Upload code to GitHub.

```
git push
```

6. git pull

Download latest code.

```
git pull
```

7. git clone

Copy project from GitHub.

```
git clone URL
```

GitHub Workflow

Create project

```
git init
```

```
git add .
```

```
git commit
```

Connect GitHub

```
git push
```

Short Remember

Git → Track code changes

GitHub → Store code online

add → Prepare files

commit → Save changes

push → Upload code

pull → Download code

clone → Copy project

Real Project Workflow

```
Create Project
   ↓
git init
```

```
   ↓
git add .
   ↓
git commit
   ↓
Create GitHub Repo
   ↓
git push
```

Example Full Workflow

```
git init

git add .

git commit -m "First commit"

git remote add origin URL

git push -u origin main
```

Golden Rule

Git tracks project history, and GitHub stores projects online for collaboration and sharing.

Git and GitHub are essential tools for modern software development.